

훅이 통과했다는 신호, 실제로 믿을 수 있는가

hooks 전수 목록과 기계 검사 결과를 바탕으로, 경로 격리·자료 정리·품질 판정의 경계 조건을 재현해 살펴보았다.

영실 · 기준 2213871afb7d655f411c46a34fd38bb8153278fc · main

수정 중인 설정을 포함한 현재 파일 복제본 기준 · 운영 코드 수정 없음

CLAIM

핵심 시험은 통과했지만,
추가 경계 조건에서 11건을 재현했다.

P1 5건은 경로 격리, 타 세션 자료 보존, 차단 결과 보존에 관한 사항이다. P2 6건은 오삭제 조건, 언어별 검사 누락, 비동기 실행 및 스테이징 검증에 관한 사항이다.

601

목록화한 파일

148,058

목록에 포함된 줄 수

5 / 6

P1 / P2 개선 항목

13

핵심 시험 통과 패키지

판정의 범위. 기존 전체 소프트웨어의 안전을 보장하는 감사 의견이 아니다. 확인된 조건에서 재현한 개선 목록이며, 관련 경로의 후속 수정·출시 판단에는 P1 해결을 우선 권고한다. 특정 PR의 병합 가능 여부는 평가하지 않았다.

조사 범위와 기준

훅은 도구 실행 전후에 끼어들어 허용 여부를 판단하거나 기록·정리를 수행하는 코드다. 본체뿐 아니라 등록 설정, 실행 래퍼, CLI 연결, 배포 템플릿이 모두 맞아야 의도한 동작에 도달한다.

BASELINE-ATTRIBUTION

검사 수준을 분리했다

현재 작업 파일을 보존한 복제본 기준 커밋은 2213871afb7d. 현재 수정된 설정 파일도 복제했고 파일별 SHA-256을 기록했다. 종료 전 원본 601개를 다시 해시하여 변경 0개를 확인했다. 원본: /Users/goos/MoAI/moai-adk-go 시험 복제본: /var/folders/kt/nq2q81cn4gx3y41r7×47ggmr000 0gn/T/moai-hooks-audit-zmytrok0	463개 Go 파일: AST 구문 분석 96개 셸 파일: bash 문법 검사 305개: 시험·fixture 296개: 비시험 코드·설정 등 파일 수는 의미 검토 완료율이 아니다. 전체 목록화·검색 후 위험 경로를 심층 검증했다.
--	--

영역	포함 내용	확인 방법
핵심 본체	internal/hook 및 security·quality·mx·memo·handoff·trace	전체 파일 목록·참조/API 검색, AST, 패키지 시험, vet, 후보 재현
CLI / 연결	hook 명령, registry 구성, 설치·pre-commit·pre-push·review gate, Codex adapter/wiring	호출 경로 검색, 집중 시험, 실제 프로세스 실행
셸 / 템플릿	.claude/hooks, .git_hooks 및 내장 배포 템플릿	96개 문법 검사, 템플릿 시험, 격리된 명령 대역 재현
현재 등록 설정	Claude 공용/로컬 설정, Codex hooks.json	JSON 파싱·등록표: 총 43개 항목. Claude 20개 이벤트, Codex 8개 이벤트
경계 밖	다른 worktree, 사용자 홈의 별도 훅 구현, 외부 런타임 자체	이번 저장소 코드 조사에 포함하지 않음

601개 파일·줄 수·해시 목록 · 현재 등록표 · 원본 재확인 결과

훅의 실행 흐름

확인된 개선 항목 11건

P1은 자료 보존·경로 격리·차단 결과와 관련된 우선 개선 항목, P2는 그 외 검증 정확성과 기능 보완 항목으로 분류했다. 신뢰도는 아래 재현 조건에 한정하여 높음으로 평가한다.

ID	우선순위	개선 항목
<u>H01</u>	P1	새 파일은 심볼릭 링크를 따라 프로젝트 밖에 기록할 수 있음
<u>H02</u>	P1	최근 갱신한 다른 팀의 자료가 오래된 디렉터리 시각 때문에 삭제됨
<u>H03</u>	P1	팀 디렉터리가 없다는 이유만으로 다른 작업 목록을 즉시 삭제함
<u>H04</u>	P2	이름이 {}인 디렉터리 안의 임의 자료를 일괄 삭제함
<u>H05</u>	P1	작업 공간 이름 ../../가 기본 체크아웃 재사용으로 연결됨
<u>H06</u>	P1	동일 커밋의 두 번째 품질 검사에서 앞선 차단 결과가 사라짐
<u>H07</u>	P2	Java·Ruby 검사 실패가 성공 종료 코드로 집계됨
<u>H08</u>	P2	.cpp 파일에서 C++ 검사기가 호출되지 않음
<u>H09</u>	P2	Kotlin 프로젝트가 Java로 감지되어 검사 없이 종료됨
<u>H10</u>	P2	단발성 ConfigChange 프로세스가 비동기 검증 결과를 남기지 못함
<u>H11</u>	P2	pre-commit 형식 검사가 스테이징 내용 대신 작업 파일을 읽음

새 파일은 심볼릭 링크를 따라 프로젝트 밖에 기록할 수 있음

```
internal/hook/pre_tool.go:989
```

확인한 사례

프로젝트 내부 linked 디렉터리를 외부 디렉터리로 연결한 뒤, 아직 존재하지 않는 linked/new.txt를 검사했다. 경로 검사는 거부하지 않았고, 임시 파일은 외부 대상에 생성되었다.

왜 개선해야 하는가

EvalSymlinks가 최종 파일 부재로 실패하면 전체 경로를 원래 문자열로 되돌린다. 이미 존재하는 부모의 링크까지 해석하지 않으므로 프로젝트 경계 검사가 우회된다.

EVIDENCE · TestAuditNewFileSymlinkBoundary

```
new_symlink_file decision="" reason=""
outside_write=true
```

개선 권고

가장 가까운 기존 부모를 실제 경로로 해석한 뒤 나머지 새 경로를 결합하고, 프로젝트 및 허용된 외부 디렉터리 경계를 다시 검사한다.

완료 판정 조건

일반 신규 파일은 허용하고, 외부로 연결된 부모 아래의 신규 파일은 거부하는 대조 시험을 추가한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 관련 경로 수정·출시 전 해결 권고

도구의 OS 샌드박스까지 우회한다는 주장은 아니다. 이 finding은 MoAI 자체 경로 검사에 관한 것이다.

최근 갱신한 다른 팀의 자료가 오래된 디렉터리 시각 때문에 삭제됨

internal/hook/session_end.go:446

확인한 사례

팀 디렉터리 시각을 과거로 설정하고 config.json을 새로 기록했다. 다른 session ID와 in_progress 작업을 둔 상태에서도 GC가 팀과 작업 디렉터를 모두 삭제했다.

왜 개선해야 하는가

부모 디렉터리의 수정 시각은 기존 파일 내용의 갱신 시각과 다르다. 현재 세션의 종료가 홈 디렉터리 전체 팀을 대상으로 이 시각만 보고 정리한다.

EVIDENCE · TestAuditStateActiveTeam

```
recently_updated_team_deleted=true task_deleted=true
```

개선 권고

팀 소유 세션의 종료가 확인된 경우에만 정리하고, 세션의 생존 상태나 명시적인 임대 만료 기록을 사용한다. 확인 불가 항목은 보존한다.

완료 판정 조건

최근 갱신한 타 세션 팀과 오래된 종료 팀을 함께 만들고 전자는 보존되는지 검증한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 관련 경로 수정·출시 전 해결 권고

실제 사용자의 활성 세션을 종료하거나 자료를 삭제하지 않았다. 재현은 전용 임시 홈 디렉터리에서 수행했다.

팀 디렉터리가 없다는 이유만으로 다른 작업 목록을 즉시 삭제함

internal/hook/session_end.go:498

확인한 사례

별도 세션 이름의 tasks 디렉터리에 방금 생성한 in_progress 작업을 두고 대응하는 teams 디렉터리는 만들지 않았다. GC는 이 작업을 즉시 삭제했다.

왜 개선해야 하는가

팀 디렉터리 부재를 작업 소유권이나 종료의 증거로 간주한다. 작업의 생성 시각, 상태, 세션 소유자를 확인하지 않는다.

EVIDENCE · TestAuditStandaloneTaskGC

```
fresh_standalone_task_deleted=true
```

개선 권고

현재 세션이 생성·소유한 작업만 정리하거나, 별도 소유권 기록으로 삭제 권한과 종료 상태를 확인한다. Stat 오류와 실제 부재도 구분한다.

완료 판정 조건

새 독립 작업 목록과 타 세션 작업 목록은 보존하고, 종료가 입증된 소유 작업만 삭제하는 시험을 추가한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 관련 경로 수정·출시 전 해결 권고

정상 런타임에서 이러한 목록이 얼마나 자주 생기는지는 측정하지 않았다. 생성된 자료를 삭제하는 조건 자체를 재현했다.

이름이 같은 디렉터리 안의 임의 자료를 일괄 삭제함

```
internal/hook/session_end.go:793
```

확인한 사례

프로젝트의 `{}/user-data.txt`에 임의 자료를 기록한 뒤 정리 함수를 호출했다. 파일의 생성 주체와 관계없이 전체 디렉터리가 삭제되었다.

왜 개선해야 하는가

과거 템플릿 치환 오류의 잔재인지 확인하지 않고 디렉터리 이름만으로 `RemoveAll`을 실행한다. 링크 자체는 제외하지만 일반 사용자 디렉터리는 보호하지 못한다.

EVIDENCE · TestAuditBogusDirectoryData

```
unattributed_user_file_deleted=true
```

개선 권고

MoAI 생성 표식과 예상 하위 구조가 확인된 잔재만 정리한다. 근거가 없으면 경고를 남기고 보존한다.

완료 판정 조건

같은 이름의 사용자 디렉터리는 보존하고, 명시적으로 표식된 잔재만 정리되는지 검사한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 개선 작업에 포함 권고

특이한 디렉터리 이름을 전제로 하므로 영향 가능 범위는 제한적이다.

작업 공간 이름 ../..가 기본 체크아웃 재사용으로 연결됨

internal/hook/worktree_create.go:94

확인한 사례

빈 임시 Git 저장소에 WorktreeName="../.."를 전달했다. 핸들러가 반환한 실제 경로는 해당 저장소의 기본 체크아웃과 같았다.

왜 개선해야 하는가

이름을 경로에 바로 결합하고, 그 위치에 디렉터리만 있으면 재사용한다. 경로 경계, 정상 worktree 등록 여부, branch 일치를 확인하지 않는다.

EVIDENCE · TestAuditWorktreeTraversalReuse

```
worktree_name=../.. returned_primary_checkout=true
```

개선 권고

이름의 경로 구분자·상위 이동을 거부하고 실제 경로가 지정된 부모 아래인지 검사한다. 재사용 전 git worktree list와 소속 ·branch를 확인한다.

완료 판정 조건

../.., 경로 구분자, 외부 링크, 일반 디렉터리는 거부하고 등록된 동일 worktree만 재사용하도록 검사한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 관련 경로 수정·출시 전 해결 권고

정상 Claude가 악성 이름을 생성한다는 증거는 없다. 입력 검증과 격리 계약의 실패를 함수 경계에서 재현했다.

동일 커밋의 두 번째 품질 검사에서 앞선 차단 결과가 사라짐

`.claude/hooks/moai/sync-phase-quality-gate.sh:179`

확인한 사례

go vet와 go build가 모두 7로 종료하도록 한 동일 커밋에서 혹을 두 번 실행했다. 첫 응답에는 차단 결과가 있으나 두 번째 응답은 비어 있었다.

왜 개선해야 하는가

검사 전에 HEAD만 저장하고 다음 호출은 즉시 반환한다. 성공·실패·검사 중단을 구분하지 않으며 실패 결과도 재전달하지 않는다.

EVIDENCE · sync_sentinel

```
first_exit=0; first_stdout contains decision:block
second_exit=0; second_stdout=""
```

개선 권고

검사 결과와 상태를 함께 원자적으로 저장한다. 완료된 실패는 같은 입력에서 재전달하고 중단된 검사는 다시 수행한다. 검사 대상이 작업 트리라면 HEAD 외 대상 내용 식별자도 필요하다.

완료 판정 조건

실패→동일 입력 재호출, 중단→재호출, 수정→재호출을 각각 검사한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 관련 경로 수정·출시 전 해결 권고

실제 Claude의 이 JSON 수용 여부는 별도 검증하지 않았다. 이 항목은 혹이 생성하는 응답의 차단 결과 소실에 관한 것이다.

Java·Ruby 검사 실패가 성공 종료 코드로 집계됨

`.claude/hooks/moai/sync-phase-quality-gate.sh:243`

확인한 사례

컴파일러/검사기 대역이 호출된 사실을 기록하고 7로 종료하도록 했다. 두 언어 모두 게이트 로그에는 0과 allow가 기록되었다.

왜 개선해야 하는가

Java는 뒤의 head 성공이 파이프라인 상태가 되고, Ruby의 `find -exec ... \;`는 각 검사 실패를 전체 명령의 실패로 전달하지 않는다.

EVIDENCE · sync_java / sync_ruby

```
java: compiler_invoked=true; decision=allow; javac compile check=0
ruby: compiler_invoked=true; decision=allow; ruby syntax=0
```

개선 권고

실제 검사 명령의 종료 코드를 먼저 보존하고 출력만 나중에 제한한다. 파일별 실행은 하나라도 실패하면 실패를 반환하도록 합산한다.

완료 판정 조건

실패 대역과 성공 대역을 각각 실행해 실제 코드가 최종 판정에 반영되는지 검사한다. PHP·Scala 등 유사 분기도 같은 방식으로 검증한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 개선 작업에 포함 권고

Java·Ruby에서 직접 확인했다. 유사한 다른 언어 분기까지 실제 실행했다고 주장하지 않는다.

.cpp 파일에서 C++ 검사기가 호출되지 않음

`.claude/hooks/moai/sync-phase-quality-gate.sh:267`

확인한 사례

CMakeLists.txt와 잘못된 내용의 bad.cpp를 둔 커밋에서 실행했다. g++ 대역 호출 기록이 없는데 게이트는 검사 코드 0으로 허용했다.

왜 개선해야 하는가

find의 -o 결합 우선순위 때문에 .cpp는 왼쪽 조건만으로 일치하며 오른쪽의 -exec를 실행하지 않는다.

EVIDENCE · sync_cpp

```
compiler_invoked=false; decision=allow; g++ syntax check=0
```

개선 권고

확장자 조건 전체를 괄호로 묶고 공통 -exec를 연결한다. 검사 대상 0건은 성공 검사와 구분해 기록한다.

완료 판정 조건

.cpp와 .cc를 각각 포함한 대조 시험에서 호출 파일 목록과 실패 전달을 모두 확인한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 개선 작업에 포함 권고

이번 재현은 .cpp 단독 입력이다. 모든 C++ 빌드 시스템의 컴파일 결과를 측정한 것은 아니다.

Kotlin 프로젝트가 Java로 감지되어 검사 없이 종료됨

```
.claude/hooks/moai/sync-phase-quality-gate.sh:62
```

확인한 사례

build.gradle.kts와 Main.kt만 있는 sync 커밋에서 실행했다. kotlinc가 호출되지 않았고 게이트 로그도 생성되지 않았다.

왜 개선해야 하는가

Gradle 마커를 모두 Java로 분류한 뒤 변경 파일은 .java만 찾는다. .kt 변경은 코드 변경 0건으로 처리되어 Kotlin 분기에 도달하지 않는다.

EVIDENCE · sync_kotlin

```
compiler_invoked=false; log_exists=false; exit=0
```

개선 권고

빌드 마커와 실제 소스 언어를 함께 판정하거나 Gradle 프로젝트의 Java·Kotlin 변경을 모두 다룬다. 다중 언어 프로젝트도 명시적으로 처리한다.

완료 판정 조건

Kotlin 단독, Java 단독, 혼합 프로젝트에서 변경 파일별 검사 경로를 확인한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 개선 작업에 포함 권고

Kotlin/Gradle 실행 도구의 실제 설치나 빌드 성공 여부와 무관하게 감지 경로를 재현했다.

단발성 ConfigChange 프로세스가 비동기 검증 결과를 남기지 못함

internal/hook/config_change.go:75

확인한 사례

같은 빌드의 CLI를 별도 프로세스로 다섯 번 실행하고 잘못된 YAML을 전달했다. 다섯 번 모두 거부 경고가 없었다. 핸들러의 WaitGroup 완료를 기다리는 대조 시험에서는 거부 경고가 나타났다.

왜 개선해야 하는가

핸들러는 고루틴을 만든 직후 반환하며 검증은 20ms 지연 후 시작한다. CLI의 종료 시 trace만 기다리고 핸들러의 비동기 작업은 기다리지 않는다.

EVIDENCE · probes-async.json / TestAuditConfigAsyncControl

```
CLI: exit=0, stdout={}, rejection_observed=false (5/5)
joined control: WARN config reload rejected (async)
```

개선 권고

단발성 혹에서는 짧은 검증을 동기 수행하거나 종료 전에 제한된 범위에서 완료를 기다린다. 긴 작업은 지속 프로세스에 전달하고 접수·완료를 기록한다.

완료 판정 조건

핸들러 단위 WaitGroup 시험뿐 아니라 실제 CLI 프로세스가 종료된 뒤 기대한 결과가 남는지 검사한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 개선 작업에 포함 권고

다섯 번의 실행을 관측한 결과다. 모든 비동기 핸들러가 항상 실패한다고 일반화하지 않는다.

pre-commit 형식 검사가 스테이징 내용 대신 작업 파일을 읽음

```
internal/cli/hook_install_precommit.go:61
```

확인한 사례

형식이 맞지 않는 main.go를 스테이징하고 작업 파일만 gofmt로 수정했다. 인덱스의 내용은 여전히 형식이 맞지 않지만 hook의 gofmt 단계는 허용했다.

왜 개선해야 하는가

git diff --cached로 파일 이름을 구한 뒤 gofmt -i은 작업 트리 파일을 읽는다. 부분 스테이징 상태에서는 커밋할 내용과 검사한 내용이 다르다.

EVIDENCE · probes-index.json

```
staged_needs_formatting=true; hook_exit=0
```

개선 권고

git show :path로 인덱스 바이트를 검사한다. 패키지 단위 검사도 인덱스 스냅샷을 대상으로 하거나 검사 범위를 명확히 표시한다.

완료 판정 조건

스테이징 내용과 작업 파일을 서로 반대로 만든 양방향 대조 시험을 추가한다.

확신 범위: 높음 — 재현 조건 안에서 확인

병합 판단: 개선 작업에 포함 권고

gofmt 단계만 분리하기 위해 go와 moai는 성공 대역으로 두었다. 전체 heavy gate가 이 커밋을 반드시 허용한다는 주장은 아니다.

H06~H09의 셀 게이트는 현재 프로젝트와 배포 템플릿이 바이트 단위로 동일하다. 후속 수정 시 두 파일과 관련 시험을 함께 맞춰야 한다. 확인값: sync_gate_template_byte_equal=true.

실행한 시험과 원본 근거

시험의 통과는 실행한 조건 안에서의 결과다. 아래에 명령과 실제 출력, 실패한 시도까지 함께 남겼다. 별도 경계 시험의 PASS는 “결함 이 없었다”가 아니라 “기록한 현상이 재현되었다”는 뜻이다.

핵심 패키지 시험

실행 위치: /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0

```
unset CLAUDE_PROJECT_DIR CLAUDE_SESSION_ID CODEX_THREAD_ID MOAI_SESSION_ID MOAI_PROJECT_DIR && go test -p 2
-count=1 -timeout 180s ./internal/hook/... ./internal/codexadapter/... ./internal/codexwiring/...
```

```
ok      github.com/modu-ai/moai-adk/internal/hook      61.499s
ok      github.com/modu-ai/moai-adk/internal/hook/handoff  1.146s
ok      github.com/modu-ai/moai-adk/internal/hook/memo  0.347s
ok      github.com/modu-ai/moai-adk/internal/hook/memo/taxonomy 0.380s
ok      github.com/modu-ai/moai-adk/internal/hook/mx      18.705s
ok      github.com/modu-ai/moai-adk/internal/hook/mx/complexity 0.474s
ok      github.com/modu-ai/moai-adk/internal/hook/perf   17.323s
ok      github.com/modu-ai/moai-adk/internal/hook/quality  13.180s
ok      github.com/modu-ai/moai-adk/internal/hook/security  7.034s
ok      github.com/modu-ai/moai-adk/internal/hook/testutil  0.465s
ok      github.com/modu-ai/moai-adk/internal/hook/trace  0.538s
ok      github.com/modu-ai/moai-adk/internal/codexadapter  0.846s
ok      github.com/modu-ai/moai-adk/internal/codexwiring  1.356s
```

전체 원본 기록

Go 구문 분석

실행 위치: /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0

```
go run /tmp/moai-hook-ast-01a08e35.go /Users/goos/MoAI/moai-adk-go/reports/hooks-audit-20260911-01a08e35/inventory.json /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0
```

```
go_ast_files=463 parse_errors=0 functions=3917
```

전체 원본 기록

셸 문법 검사

실행 위치: /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0

python3로 inventory의 셸 파일마다 subprocess.run(["bash", "-n", snapshot_path]) 실행

```
bash_syntax_files=96 failures=0
```

전체 원본 기록

정적 분석

실행 위치: /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0

```
unset CLAUDE_PROJECT_DIR CLAUDE_SESSION_ID CODEX_THREAD_ID MOAI_SESSION_ID MOAI_PROJECT_DIR && go vet -p 2 ./internal/hook/... ./internal/codexadapter/... ./internal/codexwiring/...
```

```
vet_exit=0
```

```
(vet stdout/stderr: empty)
```

전체 원본 기록

CLI 집중 시험

실행 위치: /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0

```
unset CLAUDE_PROJECT_DIR CLAUDE_SESSION_ID CODEX_THREAD_ID MOAI_SESSION_ID MOAI_PROJECT_DIR && go test -p 2
-count=1 -timeout 120s ./internal/cli -run
'^Test(HookCmd|HookHarness|HookProtocol|PrePushHookConventionBlockPlacement|MultiReviewGate|CodexHarness|Co
dexReviewGate)'
```

```
ok      github.com/modu-ai/moai-adk/internal/cli      1.014s
```

전체 원본 기록

넓은 통합 시험 — 미통과

실행 위치: /var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0

```
unset CLAUDE_PROJECT_DIR CLAUDE_SESSION_ID CODEX_THREAD_ID MOAI_SESSION_ID MOAI_PROJECT_DIR && go test -p 2
-count=1 -timeout 180s ./internal/cli ./internal/template -run
'Hook|PreCommit|PrePush|ReviewGate|StopGoal|Codex'
```

```
--- FAIL: TestPreCommitLegacyNoRecord (0.52s)
    --- FAIL: TestPreCommitLegacyNoRecord/c_no_record_pinned_released_body (0.04s)
        hook_install_precommit_attribution_test.go:259: cannot read the pinned released hook body
        "v3.1.2:internal/template/templates/.git_hooks/pre-commit" from git in
        /private/var/folders/kt/nq2q81cn4gx3y41r7x47ggmr0000gn/T/moai-hooks-audit-zmytrok0: exit status 128
        AC-PCP-005 sub-case (c) is red until it runs - a skip here would report an unmeasured
        population as a pass. Fetch tags (fetch-depth: 0) and re-run.
        panic: test timed out after 3m0s
        running tests:
            TestPreCommitNoSilentReplacement (0s)
        FAIL    github.com/modu-ai/moai-adk/internal/cli      181.353s
        ok     github.com/modu-ai/moai-adk/internal/template  2.304s
        FAIL
```

전체 원본 기록

재현 근거

Go 경계 조건 5개

셀 판정 5개 사례

실제 CLI 비동기 관측

확인한 비결함·유보 사항

핵심 13개 패키지 시험, vet, 구문 검사에서는 오류가 없었다. 현재 Codex PreToolUse 등록은 직접 CLI 한 항

완료 대기 대조 시험
스테이징 내용 대조
Go 탐침 소스 · 셸 탐침 소스

목이며, 과거의 래퍼 중복 등록 문제를 현재 결함으로 재 사용하지 않았다.

미등록 핸들러도 opt-in 경로가 존재할 수 있으므로 미 사용이라고 단정하거나 삭제를 권고하지 않았다.

초기 worktree 탐침은 macOS의 /var → /private/var 차이로 경로 문자열 비교가 실패했다. 실제 경로 비교로 시험만 바로잡아 재실행 했다. 초기 로그도 probes-go.log에 보존했다.

개선 순서와 완료 조건

새 구조를 크게 추가하기보다, 실패를 만든 조건을 가장 작은 범위에서 바로잡고 그 경계에 회귀 시험을 두는 순서를 권고한다.

HIGH

자료 보존과 격리부터

H01~H05 경로 경계와 worktree 재사용 조건을 보강한 다. H02~H03은 타 세션 자료의 소유권과 종료 증거가 확인되지 않으면 보존하도록 한다. H06은 캐시가 실패 결과를 지우지 않도록 수정한다.

MEDIUM

검사 결과와 실행 완료 보장

H04 이름만으로 삭제하는 조건을 제한한다.
H07~H09는 검사 대상과 종료 코드를 바로잡는다.
H10은 실제 프로세스 종료를 포함한 검증, H11은 인덱 스 내용을 대상으로 한 검증을 추가한다.

검토자가 직접 확인할 것. 실패 대역의 종료 코드가 최종 차단에 도달하는가? 타 세션 자료가 그대로 남는가? 실제 CLI가 종료 된 뒤 기록이 남는가? 스테이징한 바이트와 검사한 바이트가 같은가?

검증하지 못한 범위와 잔여 위험

Gaps

601개 파일을 목록화하고 전체 내용 기반 참조-위험 API 검색을 수행했다. Go 463개는 AST 구문 분석, 셸 96개는 문법 검 사 대상으로 삼았다. 의미·행동 심층 검증은 아래 11개 후보 및 실행한 시험 경로에 집중했다. 601개 파일의 모든 행·분기·호출 조합을 수동 검토하거나 실행했다는 뜻은 아니다. 저장소 전체 시험, 전체 race/coverage, Windows-Linux 실제 실행, 현재 Claude-Codex 앱의 이벤트 수용과 실제 사용자 자료 손실은 검증하지 않았다.

넓은 CLI 시험은 복제본에 v3.1.2 태그가 없어 일부 실패했고, 180초 제한에도 도달했다. 환경·시간 제한 결과를 제품 결함으 로 귀속하지 않았다. 이후 집중 CLI 시험의 통과가 해당 전체 묶음의 통과를 대신하지 않는다.

Residual-risk

합성 입력에서 재현한 동작이므로 실제 장애 빈도, 영향받은 사용자 수, 수정 후 효과는 알 수 없다. 권고를 구현한 것은 아니며, 전체 코드에 추가 결함이 없다는 보증도 아니다. 실행 부하의 원인을 분리 측정하지 않아 성능 저하 수치나 최적화 효과를 주장하지 않는다.

보고서 작성 중 저장소 HEAD는 동일했고 조사 파일 601개 해시도 모두 동일했다. 보고서·탐침·실행 기록만 새 보고서 디렉터리에 작성했다.

화면 검증: 1440·768·390px 및 390px 오프라인에서 documentWidth가 viewport와 같았고, 중복 ID·깨진 앵커·페이지 오류는 없었다. 브라우저 측정 기록 · HTML 파일 검증 · 인쇄용 PDF

2026-09-11 · 영실 · mode: incident · audience: basic — .claude/settings.local.json의 MoAI-Easy에서 결정

간결한 Markdown 원본 · 개선 항목 데이터 · 전수 목록

본문·근거는 단일 HTML에 포함되어 있다. 외부 글꼴·도식 라이브러리가 없어도 시스템 글꼴과 본문 설명으로 읽을 수 있다. 원본 로그는 동봉 파일이다.